Q1. Handle Missing Values (Basic Cleaning)

Fill missing values using the following:

Age → Median

Salary → Mean

Hours_Worked_Per_Week → Median

Performance_Score → Mean

Display the dataset after handling missing values.

In [1]:
```
import pandas as pd
```

In [2]:
```
df = pd.read_csv("employee_productivity_dataset.csv")
```

In [3]:
```
print("Missing Values Before Cleaning:")
print(df.isnull().sum())
```

```
Missing Values Before Cleaning:
Employee_ID               0
Name                      0
Department                0
Age                      25
Gender                    0
Work_Mode                 0
Join_Date                 0
Hours_Worked_Per_Week    50
Projects_Completed        0
Performance_Score        50
Salary                   25
Location                  0
dtype: int64
```

In [4]:
```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

In [5]:
```
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
```

In [6]:
```
df["Hours_Worked_Per_Week"] = df["Hours_Worked_Per_Week"].fillna(
    df["Hours_Worked_Per_Week"].median()
)
```

In [7]:
```
df["Performance_Score"] = df["Performance_Score"].fillna(
    df["Performance_Score"].mean()
)
```

In [8]:
```
print("Missing Values After Cleaning:")
print(df.isnull().sum())
```

```
Missing Values After Cleaning:
Employee_ID              0
Name                     0
Department               0
Age                      0
Gender                   0
Work_Mode                0
Join_Date                0
Hours_Worked_Per_Week    0
Projects_Completed       0
Performance_Score        0
Salary                   0
Location                 0
dtype: int64
```

In [9]:
```
print("Dataset After Handling Missing Values:")
print(df.head())
```

```
Dataset After Handling Missing Values:
  Employee_ID         Name Department   Age  Gender Work_Mode   Join_Date  \
0        E001  Amit Sharma         IT  25.0    Male    Remote  15-01-2021
1        E002   Riya Patel         HR  30.0  Female    Hybrid  15-02-2020
2        E003  John Mathew    Finance  29.0    Male    Onsite  10-03-2022
3        E004  Sneha Joshi         IT  28.0  Female    Remote  22-04-2021
4        E005  Arjun Verma  Marketing  35.0    Male    Hybrid  18-05-2020

   Hours_Worked_Per_Week  Projects_Completed  Performance_Score        Salary  \
0                   40.0                   5             4.5000  50000.000000
1                   38.0                   4             4.0000  45000.000000
2                   45.0                   6             4.3125  60000.000000
3                   41.0                   5             4.2000  52000.000000
4                   50.0                   7             4.8000  50666.666667

    Location
0       Pune
1     Mumbai
2  Bangalore
3      Delhi
4       Pune
```

Q2. Label Encoding

Convert these columns into numeric values using Label Encoding:

Gender

Department

Show the updated columns.

In [10]:
```
from sklearn.preprocessing import LabelEncoder
```

In [11]:
```
le = LabelEncoder()
```

In [12]:
```
df["Gender"] = le.fit_transform(df["Gender"])
```

In [13]:
```
df["Department"] = le.fit_transform(df["Department"])
```

In [14]:
```
print("Updated Gender and Department Columns:")
print(df[["Gender", "Department"]].head())
```

```
Updated Gender and Department Columns:
   Gender  Department
0       1           2
1       0           1
2       1           0
3       0           2
4       1           3
```

Q3. One-Hot Encoding

Apply One-Hot Encoding on:

Work_Mode

Location

Display the dataset and check how many new columns are created

In [15]:
```
original_columns = df.shape[1]
```

In [16]:
```
df = pd.get_dummies(df, columns=["Work_Mode", "Location"], dtype=int)
```

In [17]:
```
new_columns = df.shape[1]
columns_created = new_columns - original_columns
```

In [18]:
```
print("Dataset After One-Hot Encoding:")
print(df.head())
```

```
Dataset After One-Hot Encoding:
  Employee_ID         Name  Department   Age  Gender   Join_Date  \
0        E001  Amit Sharma           2  25.0       1  15-01-2021
1        E002   Riya Patel           1  30.0       0  15-02-2020
2        E003  John Mathew           0  29.0       1  10-03-2022
3        E004  Sneha Joshi           2  28.0       0  22-04-2021
4        E005  Arjun Verma           3  35.0       1  18-05-2020

   Hours_Worked_Per_Week  Projects_Completed  Performance_Score        Salary  \
0                   40.0                   5             4.5000  50000.000000
1                   38.0                   4             4.0000  45000.000000
2                   45.0                   6             4.3125  60000.000000
3                   41.0                   5             4.2000  52000.000000
4                   50.0                   7             4.8000  50666.666667

   Work_Mode_Hybrid  Work_Mode_Onsite  Work_Mode_Remote  Location_Bangalore  \
0                 0                 0                 1                   0
1                 1                 0                 0                   0
2                 0                 1                 0                   1
3                 0                 0                 1                   0
4                 1                 0                 0                   0

   Location_Delhi  Location_Mumbai  Location_Pune
0               0                0              1
1               0                1              0
2               0                0              0
3               1                0              0
4               0                0              1
```

In [19]:
```
print("Number of New Columns Created:", columns_created)
```

```
Number of New Columns Created: 5
```

In [20]:
```
print("Newly Created Columns:")
print([col for col in df.columns if col.startswith("Work_Mode_") or col.startswith("Location_")])
```

```
Newly Created Columns:
['Work_Mode_Hybrid', 'Work_Mode_Onsite', 'Work_Mode_Remote', 'Location_Bangalore', 'Location_Delhi', 'Location_Mumbai', 'Locati
on_Pune']
```

Q4. Normalization (Min-Max Scaling)

Normalize the following columns:

Salary

Hours_Worked_Per_Week

Use MinMaxScaler and display results.

In [21]:
```
from sklearn.preprocessing import MinMaxScaler
```

In [22]:
```
scaler = MinMaxScaler()
```

In [23]:
```
df[["Salary", "Hours_Worked_Per_Week"]] = scaler.fit_transform(
    df[["Salary", "Hours_Worked_Per_Week"]]
)
```

In [24]:
```
print("Normalized Columns:")
print(df[["Salary", "Hours_Worked_Per_Week"]].head())
```

```
Normalized Columns:
     Salary  Hours_Worked_Per_Week
0  0.347826               0.285714
1  0.130435               0.142857
2  0.782609               0.642857
3  0.434783               0.357143
4  0.376812               1.000000
```

In [25]:
```
print("Dataset After Normalization:")
print(df.head())
```

```
Dataset After Normalization:
  Employee_ID         Name  Department   Age  Gender   Join_Date  \
0        E001  Amit Sharma           2  25.0       1  15-01-2021
1        E002   Riya Patel           1  30.0       0  15-02-2020
2        E003  John Mathew           0  29.0       1  10-03-2022
3        E004  Sneha Joshi           2  28.0       0  22-04-2021
4        E005  Arjun Verma           3  35.0       1  18-05-2020

   Hours_Worked_Per_Week  Projects_Completed  Performance_Score    Salary  \
0               0.285714                   5             4.5000  0.347826
1               0.142857                   4             4.0000  0.130435
2               0.642857                   6             4.3125  0.782609
3               0.357143                   5             4.2000  0.434783
4               1.000000                   7             4.8000  0.376812

   Work_Mode_Hybrid  Work_Mode_Onsite  Work_Mode_Remote  Location_Bangalore  \
0                 0                 0                 1                   0
1                 1                 0                 0                   0
2                 0                 1                 0                   1
3                 0                 0                 1                   0
4                 1                 0                 0                   0

   Location_Delhi  Location_Mumbai  Location_Pune
0               0                0              1
1               0                1              0
2               0                0              0
3               1                0              0
4               0                0              1
```

Q5. Standardization (Scaling)

Apply StandardScaler on:

Age

Projects_Completed

Display the transformed values.

In [26]:
```
from sklearn.preprocessing import StandardScaler
```

In [27]:
```
scaler = StandardScaler()
```

In [28]:
```
df[["Age", "Projects_Completed"]] = scaler.fit_transform(
    df[["Age", "Projects_Completed"]]
)
```

In [29]:
```
print("Standardized Values:")
print(df[["Age", "Projects_Completed"]].head())
```

```
Standardized Values:
        Age  Projects_Completed
0 -1.172018           -0.211604
1  0.000000           -0.916949
2 -0.234404            0.493742
3 -0.468807           -0.211604
4  1.172018            1.199088
```

In [30]:
```
print("Summary Statistics After Standardization:")
print(df[["Age", "Projects_Completed"]].describe())
```

```
Summary Statistics After Standardization:
                Age  Projects_Completed
count  2.500000e+02        2.500000e+02
mean   1.598721e-17        5.995204e-17
std    1.002006e+00        1.002006e+00
min   -1.172018e+00       -1.622295e+00
25%   -7.032108e-01       -9.169493e-01
50%   -2.344036e-01       -2.116037e-01
75%    2.344036e-01        4.937419e-01
max    2.344036e+00        1.904433e+00
```

Q6. Compare Scaling Methods

Apply both:

MinMaxScaler

StandardScaler

On the Salary column.

Show both results side by side

In [32]:
```
minmax_scaler = MinMaxScaler()
standard_scaler = StandardScaler()
```

In [33]:
```
salary_minmax = minmax_scaler.fit_transform(df[["Salary"]])
salary_standard = standard_scaler.fit_transform(df[["Salary"]])
```

In [34]:
```
salary_comparison = pd.DataFrame({
    "Original_Salary": df["Salary"],
    "MinMax_Scaled": salary_minmax.flatten(),
    "Standard_Scaled": salary_standard.flatten()
})
```

In [35]:
```
print("Salary Scaling Comparison:")
print(salary_comparison.head())
```

```
Salary Scaling Comparison:
   Original_Salary  MinMax_Scaled  Standard_Scaled
0         0.347826       0.347826    -9.745089e-02
1         0.130435       0.130435    -8.283326e-01
2         0.782609       0.782609     1.364312e+00
3         0.434783       0.434783     1.949018e-01
4         0.376812       0.376812    -3.732632e-16
```

Q7. Build Preprocessing Pipeline

Create a pipeline that:

Applies encoding to categorical columns

Applies scaling to numerical columns

Use:

ColumnTransformer

Pipeline

In [36]:
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

In [44]:
```
categorical_cols = []
```

In [46]:
```
numerical_cols = [
    "Age",
    "Hours_Worked_Per_Week",
    "Projects_Completed",
    "Performance_Score",
    "Salary"
]
```

In [47]:
```
categorical_transformer = OneHotEncoder(handle_unknown="ignore")
```

In [48]:
```
numerical_transformer = StandardScaler()
```

In [50]:
```
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", categorical_transformer, categorical_cols),
        ("num", numerical_transformer, numerical_cols)
    ]
)
```

In [51]:
```
pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor)
    ]
)
```

In [52]:
```
processed_data = pipeline.fit_transform(df)
```

In [53]:
```
print("Shape of Processed Data:")
print(processed_data.shape)
```

```
Shape of Processed Data:
(250, 5)
```

In [54]:
```
print("First 5 Rows of Processed Data:")
print(processed_data[:5])
```

```
First 5 Rows of Processed Data:
[[-1.17201808e+00 -4.11220496e-01 -2.11603685e-01  6.85224455e-01
  -9.74508910e-02]
 [-1.59872116e-17 -8.68132159e-01 -9.16949301e-01 -1.14204076e+00
  -8.28332574e-01]
 [-2.34403615e-01  7.31058660e-01  4.93741931e-01  0.00000000e+00
   1.36431247e+00]
 [-4.68807231e-01 -1.82764665e-01 -2.11603685e-01 -4.11134673e-01
   1.94901782e-01]
 [ 1.17201808e+00  1.87333782e+00  1.19908755e+00  1.78158358e+00
  -3.73263169e-16]]
```

Q8. Apply Pipeline

Apply the pipeline on the dataset.

Display:

Transformed dataset

Shape of final dataset

In [55]:
```
df1 = pd.read_csv("employee_productivity_dataset1.csv")
```

In [56]:
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
import pandas as pd
```

In [59]:
```
df1["Age"] = df1["Age"].fillna(df1["Age"].median())
df1["Salary"] = df1["Salary"].fillna(df1["Salary"].mean())
df1["Hours_Worked_Per_Week"] = df1["Hours_Worked_Per_Week"].fillna(
    df1["Hours_Worked_Per_Week"].median()
)
df1["Performance_Score"] = df1["Performance_Score"].fillna(
    df1["Performance_Score"].mean()
)
```

In [60]:
```
categorical_cols = ["Gender", "Department", "Work_Mode", "Location"]
```

In [61]:
```
umerical_cols = [
    "Age",
    "Hours_Worked_Per_Week",
    "Projects_Completed",
    "Performance_Score",
    "Salary"
]
```

In [62]:
```
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ("num", StandardScaler(), numerical_cols)
    ]
)

pipeline = Pipeline([
    ("preprocessor", preprocessor)
])
```

In [64]:
```
transformed_data = pipeline.fit_transform(df1)
```

In [65]:
```
print("Shape of Final Dataset:")
print(transformed_data.shape)
```

```
Shape of Final Dataset:
(250, 18)
```

In [66]:
```
feature_names = pipeline.named_steps["preprocessor"].get_feature_names_out()

transformed_df = pd.DataFrame(
    transformed_data.toarray() if hasattr(transformed_data, "toarray") else transformed_data,
    columns=feature_names
)
```

In [67]:
```
print("Transformed Dataset:")
print(transformed_df.head())
```

Transformed Dataset:

| | cat__Gender_Female | cat__Gender_Male | cat__Department_Finance | cat__Department_HR | cat__Department_IT | cat__Department_Marketing | cat__Work_Mode_Hybrid | cat__Work_Mode_Onsite | cat__Work_Mode_Remote | cat__Location_Bangalore | cat__Location_Delhi | cat__Location_Mumbai | cat__Location_Pune | num__Age | num__Hours_Worked_Per_Week | num__Projects_Completed | num__Performance_Score | num__Salary |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 | -1.172018 | -0.411220 | -0.211604 | 0.685224 | -0.097451 |
| 1 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.000000 | -0.868132 | -0.916949 | -1.142041 | -0.828333 |
| 2 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | -0.234404 | 0.731059 | 0.493742 | 0.000000 | 1.364312 |
| 3 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | -0.468807 | -0.182765 | | | |
| 4 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 1.172018 | 1.873338 | | | |

```
3               -0.211604              -0.411135     0.194902
4                1.199088               1.781584     0.000000
```

Q9. Conceptual Question

Why is scaling important in Python?

Scaling is important because it brings numerical features to a similar range, preventing variables with larger values from dominating those with smaller values. It helps machine learning algorithms perform more accurately and efficiently, improves the speed of model training, and ensures fair comparison between features. Scaling is especially important for distance-based algorithms such as KNN, K-Means, and SVM, where differences in feature scales can significantly affect the results.

Before Scaling:

Age Salary 25 50000 30 80000

Here, Salary values are much larger than Age values.

After Scaling:

Age Salary -0.85 -1.02 0.35 0.78

Now both features are on a comparable scale.

Q10. Conceptual Question

Why do we convert categorical data into numerical form?

We convert categorical data into numerical form because most machine learning algorithms can only work with numerical values and cannot directly understand text-based categories such as "Male," "Female," or "HR." Encoding techniques like Label Encoding and One-Hot Encoding transform these categories into numbers, allowing algorithms to process the data effectively. This helps improve model performance and ensures that categorical information can be used in analysis and predictions.